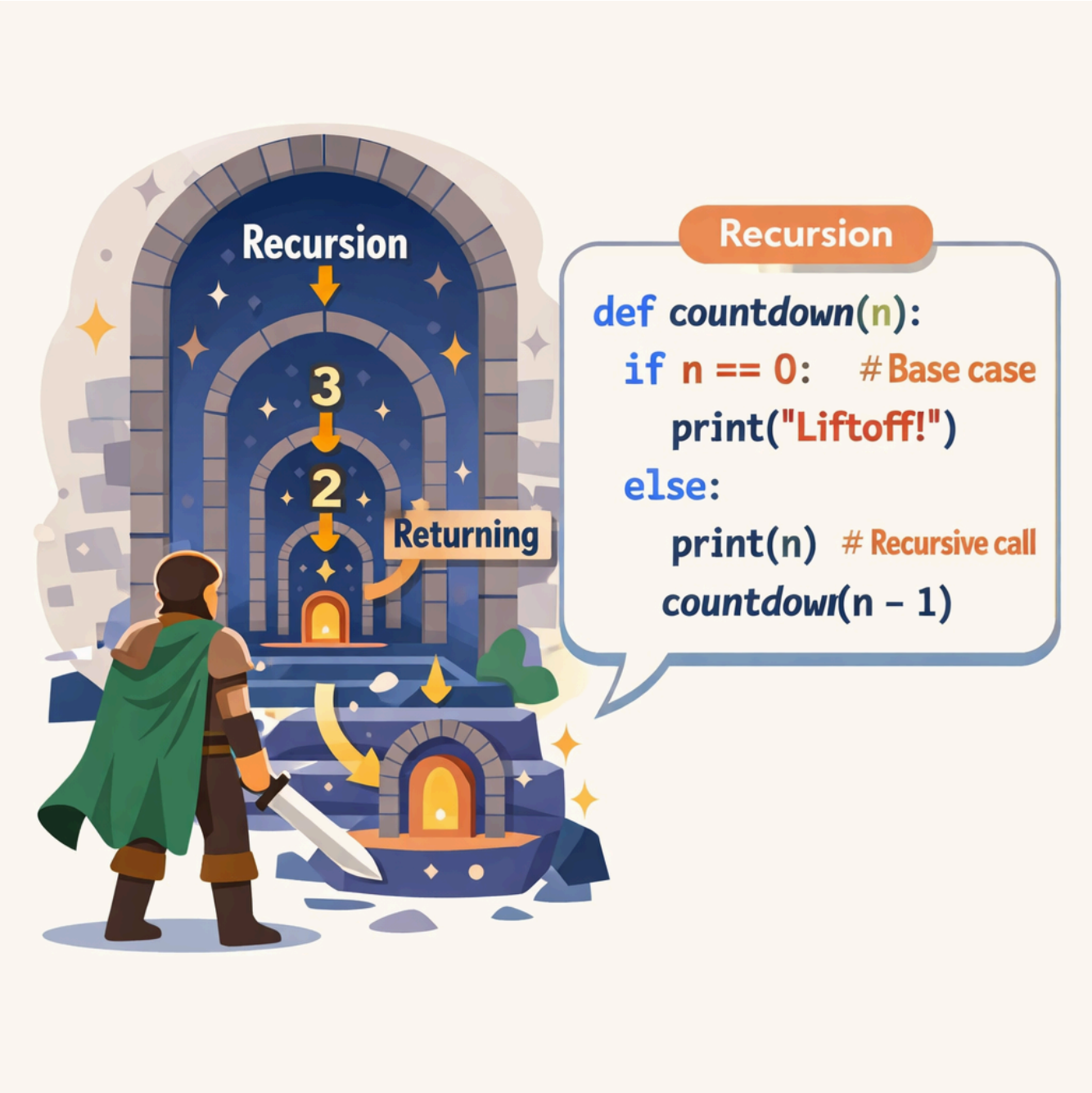
```
from IPython.core.display import display, HTML
display(HTML("<style>.rendered_html table { font-size: 17px; }</style>"))
```

Recursion



It is legal for one function to call another; it is also legal for a function to call itself. It may not be obvious why that is a good thing, but it turns out to be one of the most magical things a program can do.

For example, take a look at the following program:

```
def countdown(n):
  if n <= 0:
    print('Blastoff!')
  else:
```

```
    print(n)
    countdown(n-1)
```

What happens if we call this function as `countdown(3)`?

- The execution of countdown begins with `n=3`, and since n is greater than 0, it outputs the value 3, and then calls itself...
  - The execution of countdown begins with `n=2`, and since n is greater than 0, it outputs the value 2, and then calls itself...
    - The execution of countdown begins with `n=1`, and since n is greater than 0, it outputs the value 1, and then calls itself...
      - The execution of countdown begins with `n=0`, and since n is not greater than 0, it outputs the word, "Blastoff!" and then returns.
      - The countdown that got `n=1` returns.
    - The countdown that got `n=2` returns.
  - The countdown that got `n=3` returns.
- And then we go back in the beginning.

So, the total output looks like this:

3

2

1

Blastoff!

Example: Consider a function that prints numbers from a given number `n > 0` up to `0` in reverse order. This function can achieve this task by calling itself recursively.

A recursive definition is made up of two parts:

- One, or more, *base case* that directly specifies the result for a special case. This is also known as *exit condition*, i.e., the function must know when to stop.
- A *recursive (inductive)* case that defines the answer in terms of the answer to the question on some other input, typically a simpler version of the same problem.

Factorials

One well-known recursive definition is probably the factorial function on natural numbers (`x!`)

The classic inductive definition is:

- $1! = 1 \rightarrow$ **base case**
- $(n+1)! = (n+1) * n! \rightarrow$ **inductive case**

Let's see two approaches of calculating the factorial of a (positive) integer:

- The first approach (left) uses an iterative approach
- The second approach (right) uses a recursive approach

```
def factI(n):                  def factR(n):
   result = 1                      if n == 1:
   while n > 1:                       return n
      result = result * n         else:
      n -= 1                          return n*factR(n - 1)
   return result
```

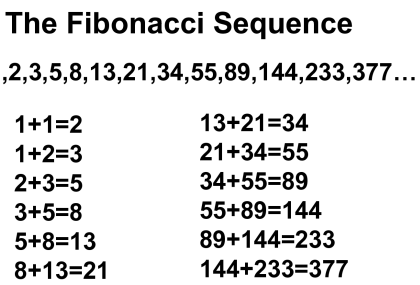
The first thing we notice is that the recursive version, unlike the iterative one, *does not* include an iteration.

- This provides a significant boost to computational times, though in cases of large inputs the recursive approach will tend to allocate more memory.
- Notice that `factR` calls, and returns, itself in the inductive case, with the original input reduced by 1.
- There is no rule as to whether the base case should form up the `if` conditional, while the inductive case being the `else` code block. Below is the same example, with the two cases reversed:

```
def factR(n):
    if n > 1:
      return n*factR(n-1)
    else:
      return n
```

Fibonacci sequence

Fibonacci number sequence is another well-known example that recursion is used to generate them.



From the image above, we can derive the recursive process, but we have to be careful when it comes to the base case:

- It has two base cases, not just one.
  - In general, you can have as many base cases as you need.
- In the recursive case, there are two recursive calls, not just one.
  - Again, there can be an arbitrary number of them, given the structure of the process.

Below is a recursive Python implementation of the Fibonacci sequence:

```
def fib(n):
    """Assumes n int >= 0
    Returns Fibonacci of n"""
    if n == 0 or n == 1:
        return 1
    else:
        return fib(n-1) + fib(n-2)

def testFib(n):
    for i in range(n+1):
        print('fib of', i, '=', fib(i))
```

Going back to the example of first-class functions from the previous lecture, we can now include the two recursive functions we discussed above.

As you can see, we can use both built-in and custom-made functions as inputs to other functions:

```
def applyToEach(L, f):
    """Assumes L is a list, f a function
    Mutates L by replacing each element, e, of L by f(e)"""
    for i in range(len(L)):
        L[i] = f(L[i])

L = [1, -2, 3.33]
print('L =', L)
print('Apply abs to each element of L.')
applyToEach(L, abs)
print('L =', L)
print('Apply int to each element of', L)
applyToEach(L, int)
```

```
print('L =', L)
print('Apply factorial to each element of', L)
applyToEach(L, factR)
print('L =', L)
print('Apply Fibonnaci to each element of', L)
applyToEach(L, fib)
print('L =', L)
```

```
import math

def factR(x):
    return math.factorial(x)

def fib(x):
    if x < 0:
        print("Incorrect input")

    # Check if n is 1,2
    # it will return 1
    elif x == 0 or x == 1:
        return 1

    else:
        return fib(x-1) + fib(x-2)

def applyToEach(L, f):
    """Assumes L is a list, f a function
    Mutates L by replacing each element, e, of L by f(e)"""
    for i in range(len(L)):
        L[i] = f(L[i])

L = [1, -2, 3.33]
print('L =', L)
print('Apply abs to each element of L.')
applyToEach(L, abs)
print('L =', L)
print('Apply int to each element of', L)
applyToEach(L, int)
print('L =', L)
print('Apply factorial to each element of', L)
applyToEach(L, factR)
print('L =', L)
print('Apply Fibonnaci to each element of', L)
applyToEach(L, fib)
print('L =', L)
```

*Performing recursion using dictionaries

Calculate individual numbers of the Fibonacci sequence using functions and dictionaries!

```
def fib_efficient(n, d):
    if n in d:
        return d[n]
    else:
        ans = fib_efficient(n-1, d) + fib_efficient(n-2, d)
        d[n] = ans
        return ans

d = {1:1, 2:2}
print(fib_efficient(6, d))
```

```
def fib_efficient(n, d):
    if n in d:
        return d[n]
    else:
        ans = fib_efficient(n-1, d) + fib_efficient(n-2, d)
        d[n] = ans
        return ans

d = {1:1, 2:2}
```

```
print(fib_efficient(6, d))
d
```

```
13
{1: 1, 2: 2, 3: 3, 4: 5, 5: 8, 6: 13}
```